Operating System Security

Week 6: Rootkits

Background to Understand Rootkits: Pointers

One of the things Rootkits tend to do is overwrite pointers.

To understand why, you need to understand what a pointer is!

If you have programmed in C or C++ before, you should know this stuff.

But this mini-lecture assumes no background in C/C++.

High-level Description: A Pointer is an Address in Memory

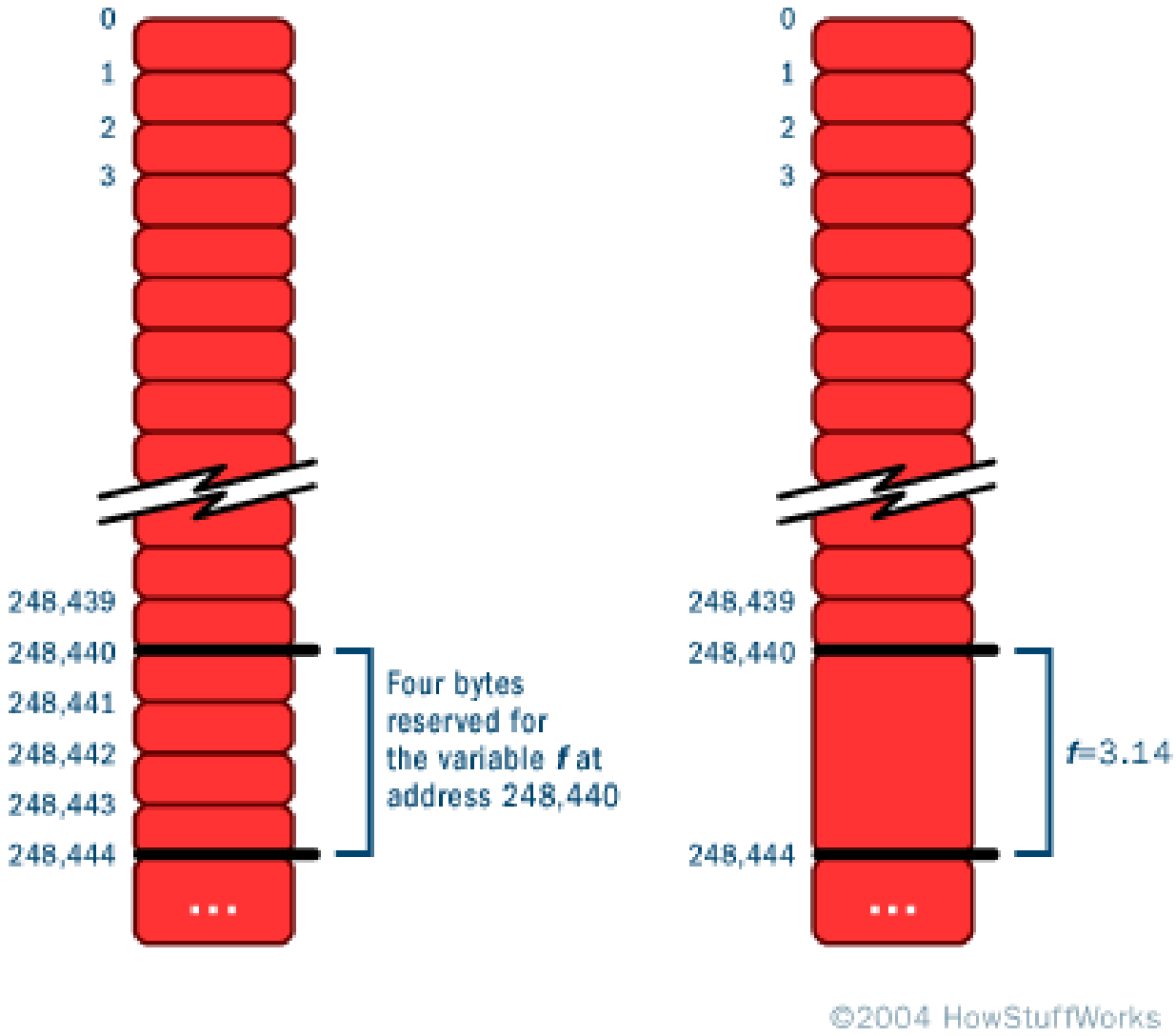
Recall that memory is an addressable array

Imagine you have a variable f, that requires 4 bytes of memory.

```
f = 3.14
```

It needs to be stored somewhere in memory (an addressable array), and in a way it can be found by the program.

The address of f is where it is stored (in the next slide, 248,440)



* Image source: “Pointers: Understanding Memory Addresses”, https://computer.howstuffworks.com/c23.htm, last accessed Feb. 13, 2026.

Another example

Now, what does it mean if the program stores the address of f in another variable?
---- that's a pointer to f.

- Pointers are just variables whose values are the addresses of other variables.

Let's say we have 2 variables

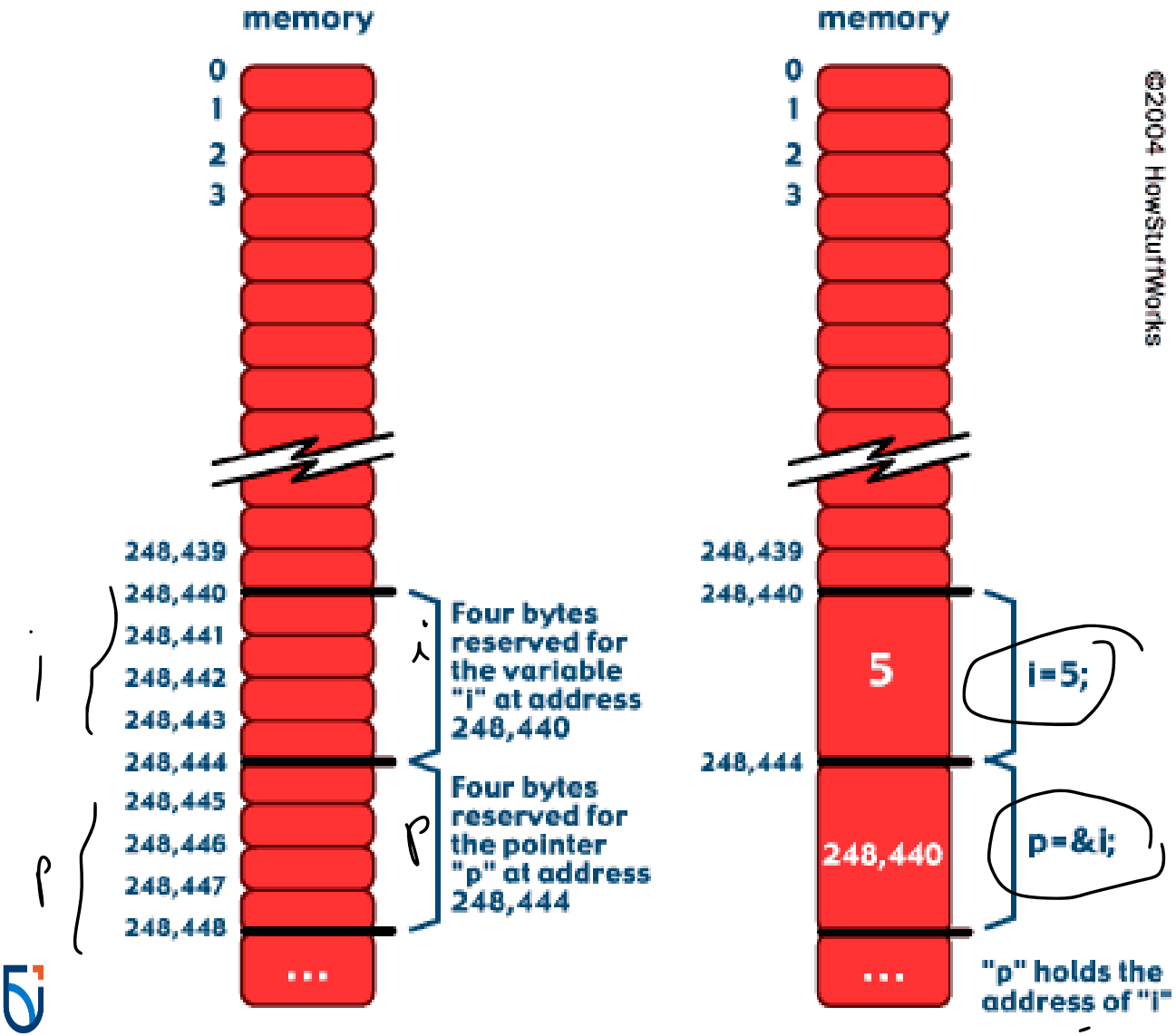
- i (located at address 248,440)
- p (located at address 248,444)

```
i = 5
p = 248440
```

Consider the *contents of the variables:*

- Could i be a pointer? What about p?

A Short Lesson on Pointers (continued)



* Image source: “Pointers: Understanding Memory Addresses”, https://computer.howstuffworks.com/c23.htm, last accessed Feb. 13, 2026.

Now consider what happens if another program updates the value of our pointer p?

In our previous example, what would happen?

It would likely impact the program's output, but how depends on what p does.

I'm dying to know how this relates to Rootkits! A hint please?

Consider: what if p contained an address to other code, which our program intended to run?

```
if (i < 10)
    p = myLowFunc
else
    p = myHighFunc
…
…
…
p()
```

Can programs just do that?

Thankfully, it's normally up to the programmer to decide if this happens, and other programs *typically* can't overwrite other program's memory.

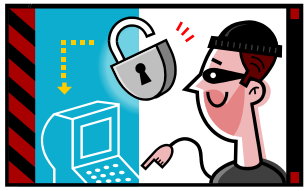
But: if that code is in the kernel, there are no access control protections!

You can imagine then how one piece of code, if installed in the kernel, can wreak havoc with the system.

OK, now you're ready to learn about Rootkits ☺

Rootkits

Rootkits are trojan horse backdoor tools that modify existing operating system software so that an attacker can gain access to and hide on a machine



User-mode rootkits (i.e., in programs running on top of the OS kernel) normally install a backdoor and try to overwrite key executables that may reveal the rootkit (e.g., ps and ls on Linux).

Kernel-mode rootkits live inside of the OS kernel. This is worse than if the rootkit were user-mode.

Source of Some of Today's Material

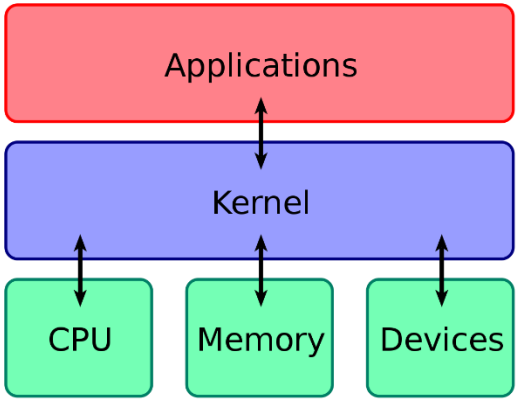
1) Skoudis et al., "Malware: Fighting Malicious Code", Chapter 8, Prentice-Hall, 2005.

- Surprisingly, some of this material is still valid today.

2) Quade, "The Spy Within", Linux Magazine, http://www.linux-magazine.com/Online/Features/Kernel-Rootkit-Tricks, last accessed Feb. 13, 2026.

3) MalwareTech, Inline Hooking for Programmers (Part 1: Introduction)", 2015, https://www.malwaretech.com/2015/01/inline-hooking-for-programmers-part-1.html, last accessed Feb. 13, 2026.

Recall: the Kernel



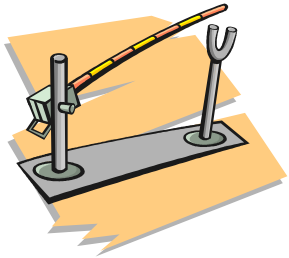
The kernel performs many/most critical operating system tasks, e.g.:

- Process and thread control
- File system control
- Memory control

The kernel for Windows and Linux runs in a different CPU sensitivity level called ring 0.

- X86-compatible CPUs have Rings 0-3, where 3 is the least sensitive.
- Windows and Linux both only use ring 0 for kernel-mode, and ring 3 for user-mode.
- Running in ring 0 provides direct access to kernel memory.

User- to Kernel-mode interaction



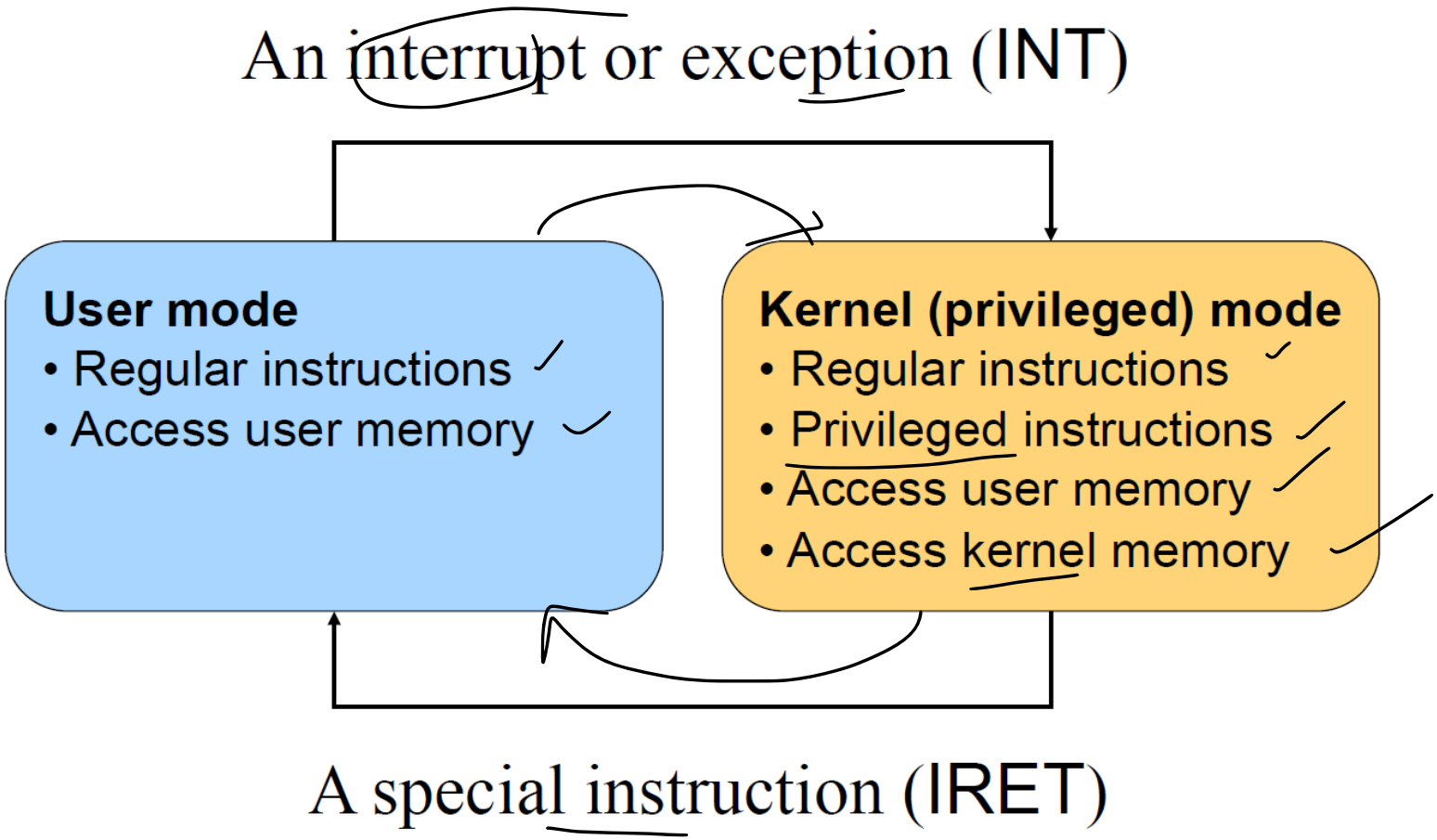
When most programs run, control sometimes must pass from user mode into kernel mode

- E.g., when the program needs to interact with hardware for receiving a packet or printing to the screen

When this happens, control is very carefully passed from user mode to kernel mode, through tightly controlled interfaces.

The software that implements this transition from ring 3 to ring 0 is called a "call gate".

Call gates: what is really happening

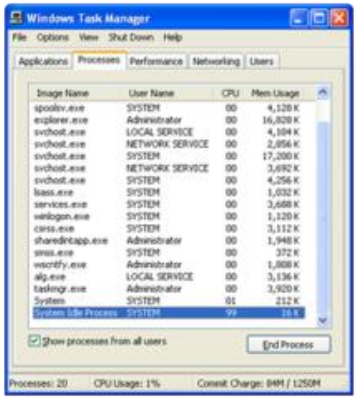


[*] Courtesy of Andy Bavier, "OS Structure and System Calls", slides 4-6, 2010, URL: https://www.cs.princeton.edu/courses/archive/fall10/cos318/lectures/OSStructure.pdf, last accessed Feb. 13, 2026.

User- to Kernel-mode interaction (con't)

When we use a process list program (e.g., using Windows task manager or Linux ps)…

- the process list program runs a command from user mode that asks the kernel to list all running processes
- The kernel grabs information from its kernel-mode data structures and generates a response for the user-mode process
- The user-mode process displays the information the kernel returned to it.



What might be possible if an attacker modified the kernel itself?

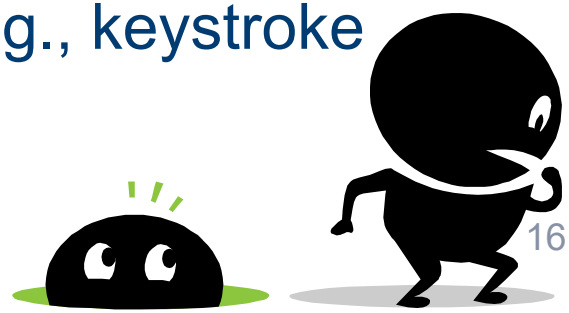
Hiding in the Kernel

The attacker can modify the kernel so it lies to user mode programs

- In our previous example, it could list all processes that are not related to e.g., the attackers backdoor process.

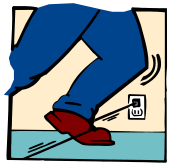
Most kernel-mode rootkits do the following:

- File and directory hiding
- Process hiding
- Network port hiding (e.g., listening ports)
- Promiscuous mode hiding
- Device interception and control (e.g., keystroke loggers)



TripWire

A tool for file integrity checking



Based on using cryptographic hashes of files, stores them in a database, and later checks they haven't changed.

Useful for files that "should not change" in e.g., the OS.

Note: this is a tool that provides an additional layer of security, not part of the OS itself.

What are the limitations of this approach?

Hiding in the Kernel (con't)

Another technique that is often found in rootkits: execution redirection.

Execution redirection is when an admin or user runs a program, the kernel swaps that program with a modified version, and executes the modified version instead.

- The admin/user cannot tell the difference
- What would an integrity checker e.g., tripwire be able to tell us?

How do Kernel Rootkits Gain Privileged Execution?

Some methods:

1. Trick superuser to install a kernel module (e.g., a driver)
2. Place a Trojan horse in a kernel module
3. Exploit a vulnerability in the kernel
4. Direct Memory Access (DMA) writes to kernel memory through hardware devices (e.g., Firewire, USB 4, …)

What do Kernel Rootkits do Once Running?

The goal is typically to insert itself within the kernel so that it can hide itself over a longer period of time. Some methods to achieve this:

1. Kernel table hooking
2. Inline hooking
3. Function hooking

Some Background: Windows System Service Dispatch & Interrupt Tables

Interrupt Handler	
1	0x7F..04
…	…
9	0x7F..9F
…	…
2E	0x7F..F0

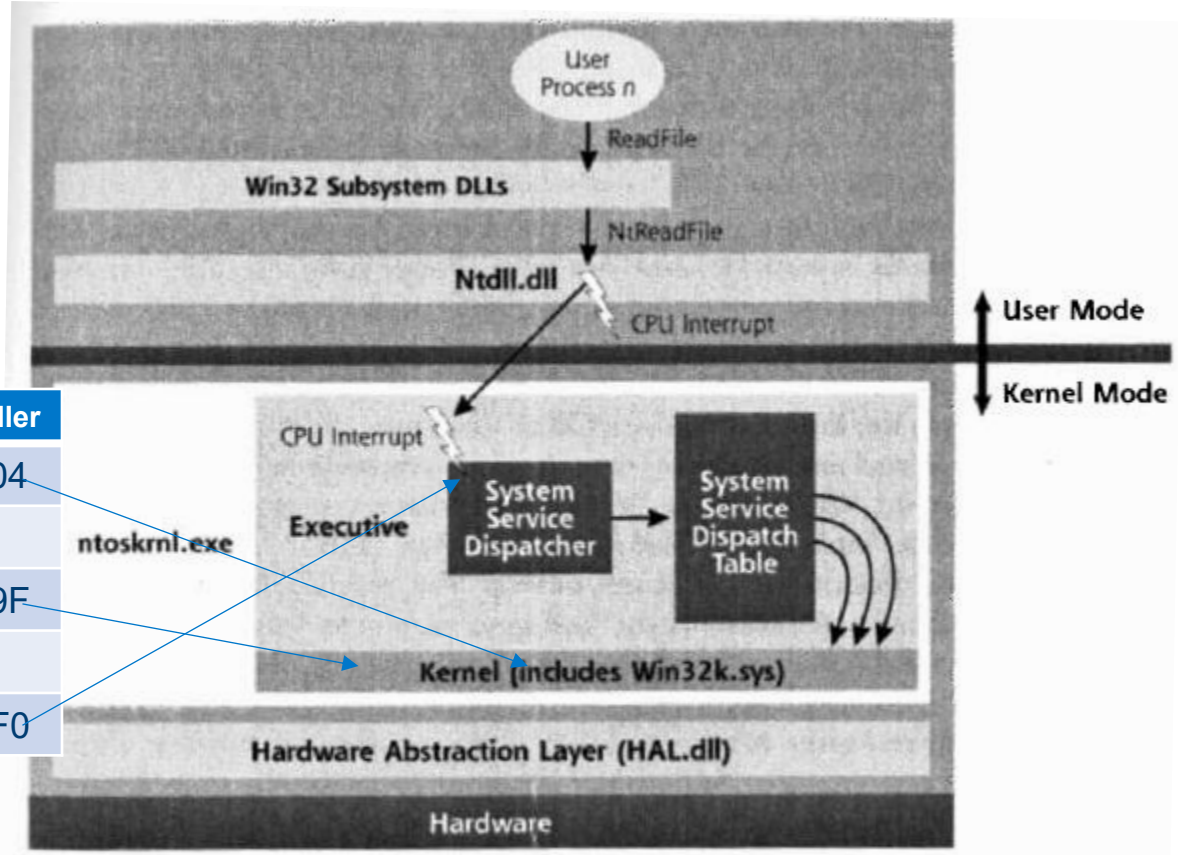


Figure 8.23
System service dispatching in Windows.

* Image from Skoudis et al., "Malware: Fighting Malicious code", Prentice Hall, 2005

Kernel Table Hooking: Windows

The attacker can ensure their code is run instead of the kernel's using one of these options:

- Alter the system service dispatch table so that it points to the attacker's code instead of the existing kernel functionality
  - (see next slide, "B" label)
- Interrupt hooking, which modifies how the kernel handles CPU interrupts.
  - By changing the table's entry 2E (which normally forwards to the System Service Dispatcher), the attacker could redirect calls to the system service dispatcher to the attacker's own code
  - (see next slide, "C" label)

Kernel Table Hooking: Windows (continued)

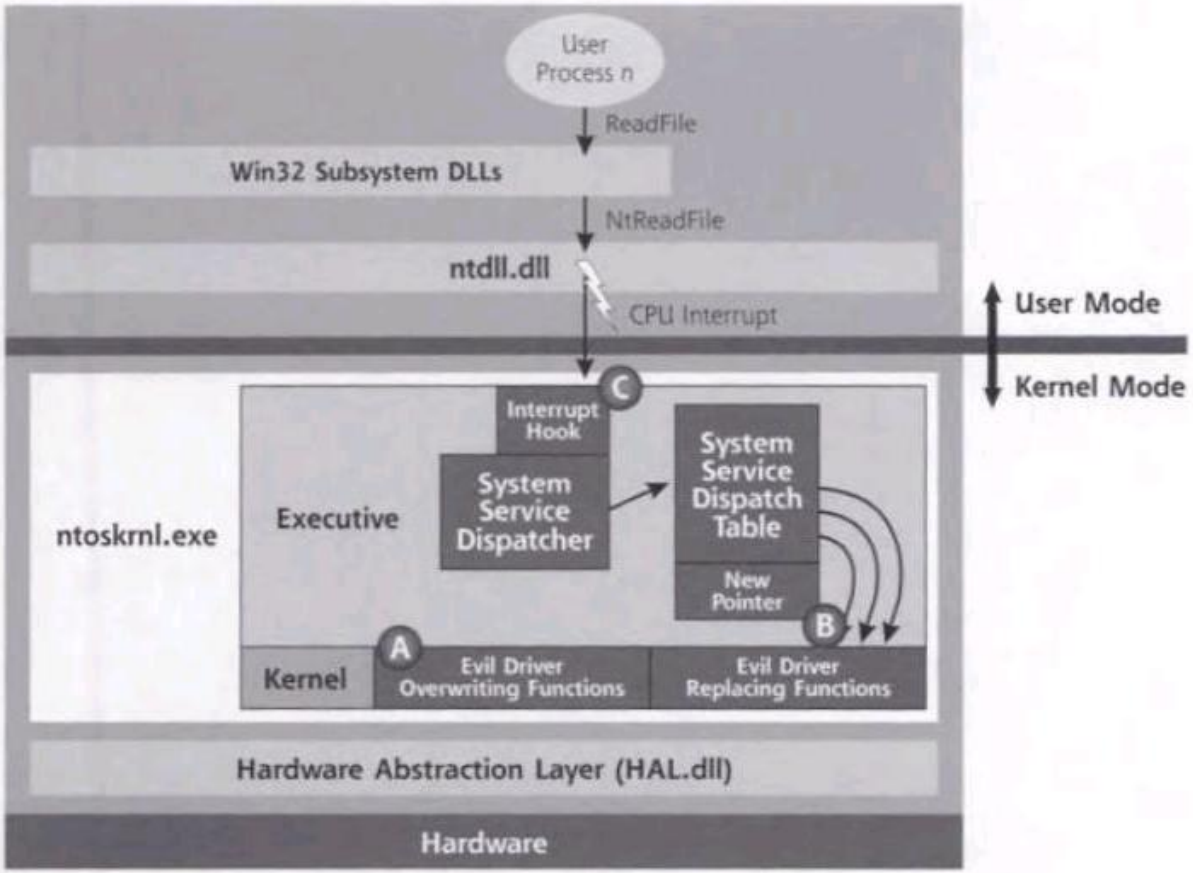


Figure 8.34
Using a device driver to manipulate the Windows kernel.

* Image from [1]

Kernel Table Hooking: Linux

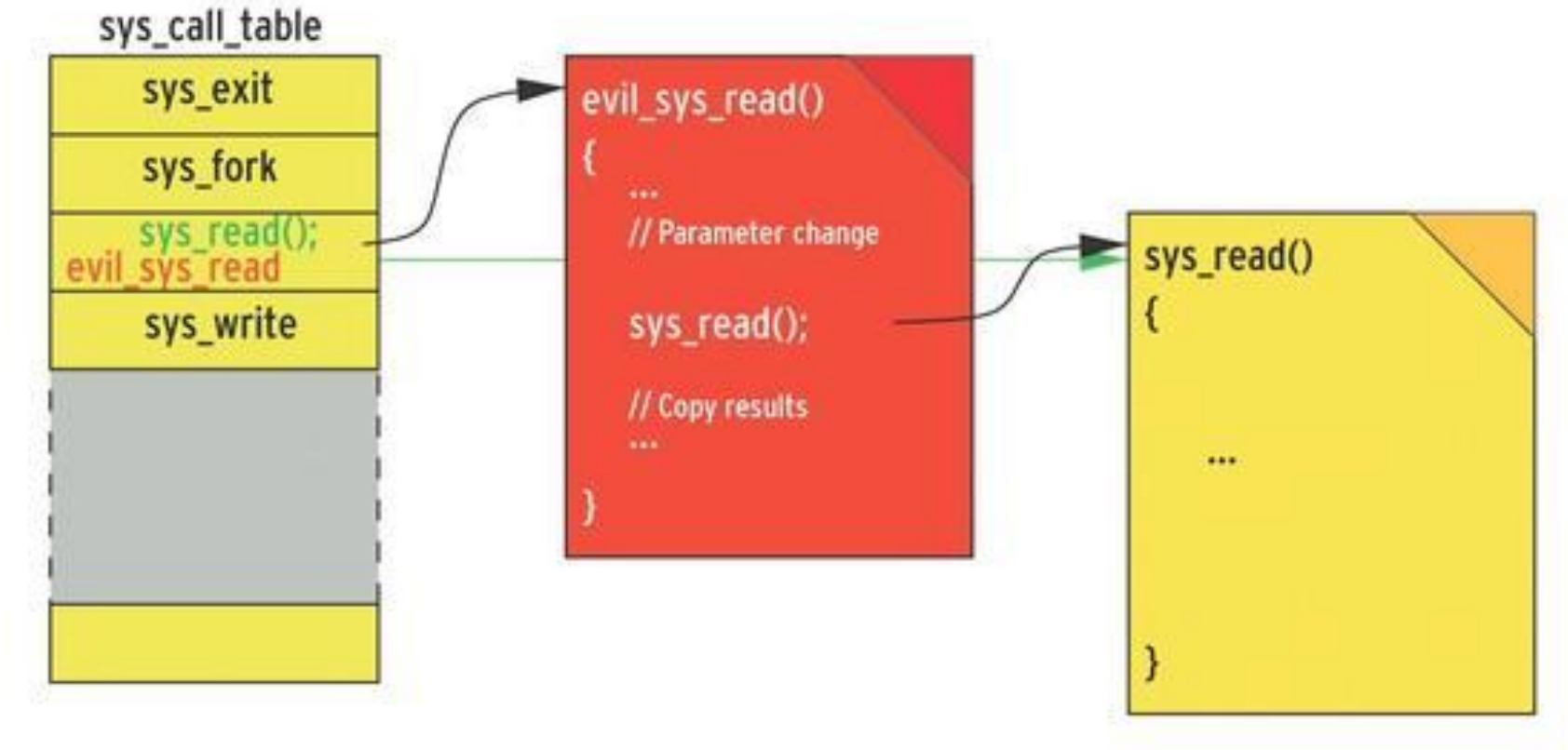


*Image from [3].

Inline Hooking

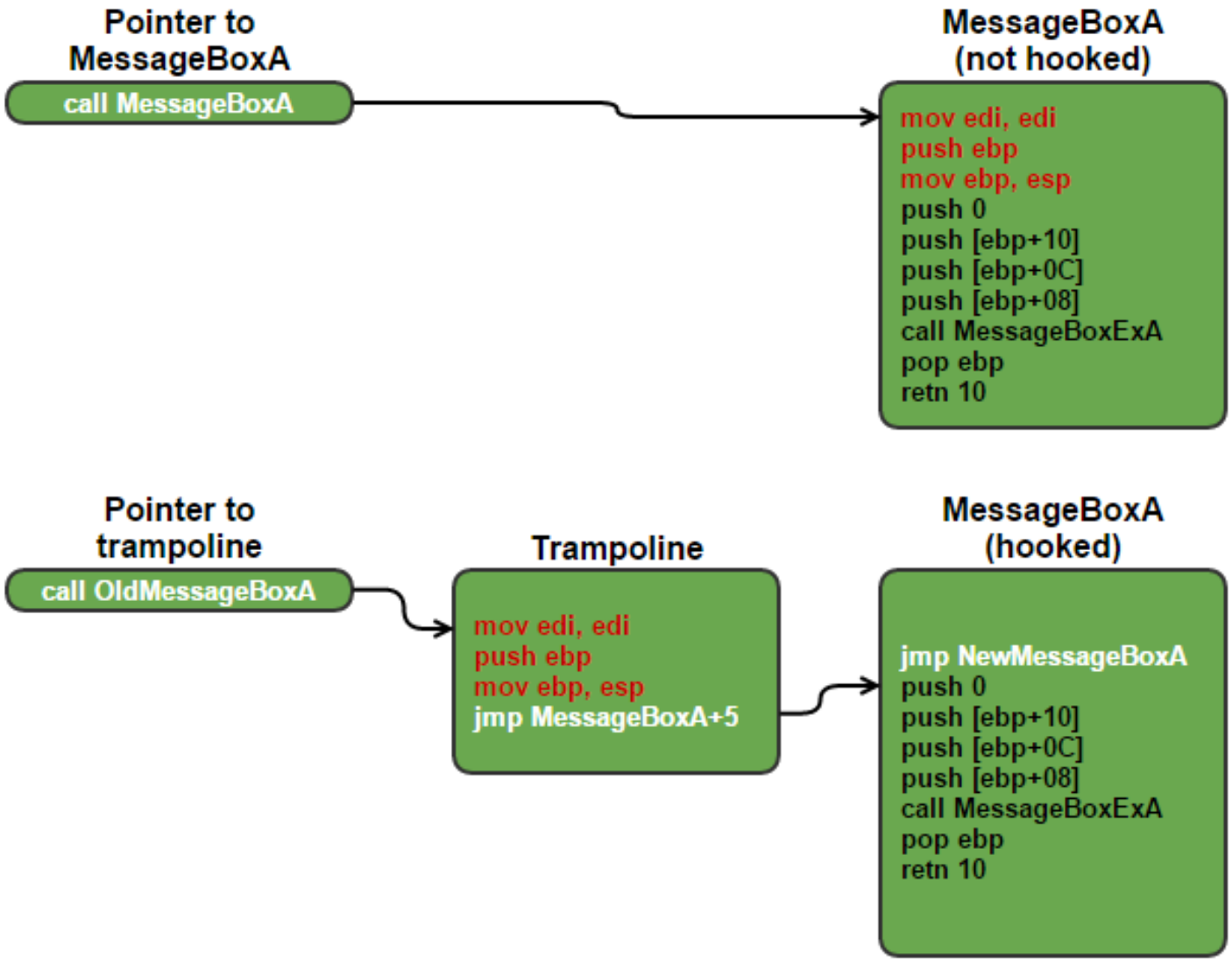
Table hooks can be detected by simple checks of the table addresses vs. stored versions.

Inline hooking: directly modifying code within the target function by overwriting the first few bytes (with a jump to attacker's code).

Can be detected by hashing code, comparing to stored version.

The change can't be written to disk on systems that use code signing

Red: Instructions we overwrite
White: Instructions we've written



Inline hook example for Windows MessageBoxA function [2].

What is the difference between inline hooking and kernel table hooking?

a) Inline hooking hooks the system service dispatch table, whereas kernel table hooking hooks the system call table.

b) Inline hooking hooks the interrupt table, whereas kernel table hooking hooks the system call table.

c) Inline hooking hooks hooks e.g., the system call table, whereas kernel table hooking overwrites the first few bytes of the function with a jump to the attacker's code.

d) Inline hooking overwrites the first few bytes of the function with a jump to the attacker's code, whereas kernel table hooking hooks e.g., the system call table.

TCB File Protection: Driver Signing

64-bit versions of Windows Vista (and later) enforce that all Microsoft kernel drivers be digitally signed

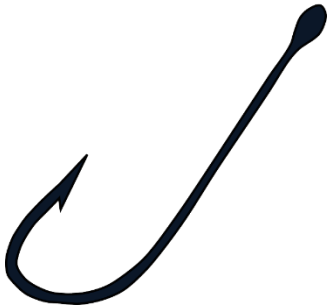
- Windows 10 version 1607 and later won't load drivers that are not signed by their Dev Portal.
- The digital signatures of these kernel drivers are then verified by the OS at load-time.
- As a result, drivers that do not have a valid signature will not be loaded into the kernel.

You can configure Linux systems to do this too! But it might not be on by default.

Function Hooking

Similar to kernel table hooking, but done in user mode instead of kernel mode

The Import Address Table (IAT) contains addresses of functions in dynamically linked libraries (DLLs).



The IAT's address for a certain function can be overwritten (hooked) here, just as for the SSDT, interrupt descriptor table, and sys_call_table

How much of the kernel needs to be overwritten for serious damage?

Generally, if a function pointer table exists somewhere in the kernel, a security-critical function pointer could be overwritten.

- e.g., in a function pointer in the system service dispatch table
- A function pointer in a 32-bit architecture is 4 bytes, 64-bit architecture is 8 bytes.
- Consider the damage if the function call was the main entry point to the reference monitor.

E.g., consider what could happen if the ***SeAccessCheck*** function, which does all access control checks on a Windows system, were overwritten?

Historical Method: Patching the Kernel on the Hard Drive

Overwrote the actual kernel executable itself.

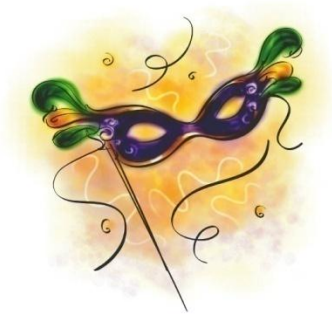
This required also overwriting the executable that starts the kernel, as it did an integrity check.

No longer an issue with kernel code signing!

Creating a fake system using a virtual machine

Basic idea: attacker installs a VM tool, builds a virtual system that mimics the original machine, then configures it to start up at boot using startup scripts.

- What can the attacker gain by doing this?



What if the administrator was observing the boot process?

Defending the Kernel Against Rootkits - Prevention

Hardening configuration to make it more difficult for attackers to install first-stage rootkit installers on the system.

- E.g., on Linux, it is possible to recompile the kernel with all of the drivers you need, and disable loadable kernel modules.

Applying patches in a timely manner!

Intrusion prevention systems – lock out functionality often abused by attackers.

- E.g., can configure the system calls a given program (e.g. a web server) can make.

Consider: how can you manage a server with disabled loadable kernel modules?

- When does it make sense to do it?

- What resources could be requested to help?

Defending the Kernel Against Rootkits - Detection

Anti-virus tools

- But really only if the anti-virus is installed and updated before the kernel-mode rootkit is installed – why?

Integrity checking tools (e.g., Tripwire)

- Although it's catching anything depends upon the attacker not doing a good job of disguising any file changes made by the rootkit

Some Tools for Rootkit Detection

Fedora Linux (included on Fedora Security Lab):

- Unhide – unhides processes and ports
- Rkhunter – A host-based tool to scan for rootkits, backdoors and local exploits
- Chrootkit – Tool to locally check for signs of a rootkit

More About Unhide – Windows Version

For Windows, a couple of utilities are:

- WinUnhide - Compare info gathered from wmic command with info gathered from openprocess and Toolhelp
  - Idea is that different APIs may not all be intercepted.

- WinUnhide-TCP - First it lists open TCP/UDP ports through GetTcpTable and GetUdpTable and then identify hidden ports using bind() bruteforcing
  - Compare API listings to actual system probing.

More About Unhide – Linux Version

Unhide -- detects hidden processes. Implements six techniques, e.g.:

- Compare /proc vs /bin/ps output
- Compare info gathered from /bin/ps with info gathered from syscalls (syscall scanning).
- Full PIDs space occupation (PIDs bruteforcing)

Unhide-TCP -- identifies TCP/UDP ports that are listening but not listed in /bin/netstat by brute forcing of all TCP/UDP ports available

- (same idea as what is done in WinUnhide-TCP)

More about rkhunter

A tool that scans for rootkits, backdoors and local exploits by running tests like:

- MD5 (or SHA-1) hash compare of important files with known good ones in online database
- Looking for default files used by rootkits
- Looking for wrong file permissions for binaries
- Looking for suspicious strings in kernel modules
- Looking for hidden files

More about chrootkit

A shell script using common UNIX/Linux tools like the strings and grep commands to

- search core system programs for signatures and
- compare a traversal of the proc filesystem with the output of the ps (process status) command to look for discrepancies.

Defending the Kernel Against Rootkits - Response

Bring a CD with a fresh copy of your antivirus tool installation and the latest signatures.

- Many antivirus tools can detect and then uninstall various kernel-mode rootkits

Live CDs

- These are Operating Systems that run on CD, so data changes are not persistent.
- Allows analysis of hard drive without the OS (and the rootkit) running.
- Can use tools included to analyze the files on the hard drive.

Summary Questions

Why are kernel-mode rootkits more concerning than user-mode rootkits?

What are the techniques that an attacker can use to hijack system calls?

What is the difference between inline hooking and hooking a function pointer (e.g., within the system call table)?

What are the best defenses against rootkits?